PPT Outline Intelligent Generation and Content Completion System Project Charter

RBS

The Responsibility Breakdown Structure (RBS) clarifies the responsibilities of each module and sub-task, ensuring that each team member's duties are clear, avoiding overlaps or ambiguities, thus optimizing project management. It helps the team define task objectives, enhance collaboration efficiency, and ensures that the project is completed on time and with high quality. Through RBS, the project can effectively monitor progress, make timely adjustments, and ensure that key modules receive sufficient attention, driving the smooth progress of the project. The project requirements are mainly focused on three aspects: structured outline generation, knowledge retrieval and content completion, and end-to-end evaluation system.

Responsibility Assignment

To ensure clear boundaries and avoid task overlaps based on our finalized technical stack (FastAPI, React, pgvector, Redis Stream), the core responsibilities are mapped to team members as follows:

Technical Patterns	Member	Core Responsibility Description
Overall Architecture & Backend Core	Ma Xiaolong	Responsible for system architecture design, FastAPI backend gateway development, and the implementation of the Session/Task asynchronous state machine.
Prompt Engineering & Model Comparison	Zhang Ruiqi	Responsible for prompt strategy optimization (including JSON Schema constraints and intent recognition) and the horizontal performance comparison of multiple models
RAG Retrieval & Data Storage	Tang Chengyang	Responsible for the RAG knowledge retrieval module (vector storage based on pgvector), document parsing logic, and the maintenance of the Redis Stream asynchronous queue.
Frontend Development & End-to-End Evaluation	Xie Jingcheng	Responsible for frontend UI development (React + Vite), SSE streaming output integration, and the construction of the automated end-to-end evaluation system.

Structured Outline Generation

This module is responsible for transforming user-input topics or lengthy documents into well-organized, multi-level PPT narrative frameworks. The generated outlines must adhere to a

four-level hierarchy—chapters, pages, key points, and notes—ensuring logical coherence, hierarchical reasonableness, and appropriate page-level granularity.

- **Prompt Engineering Strategy Research**: The project will systematically investigate the impact of various prompt engineering strategies on outline quality, including but not limited to Chain-of-Thought prompting and few-shot learning techniques. Different prompting approaches will be designed, tested, and compared to identify the optimal strategy that yields the most logically coherent and structurally sound outlines.
- **Structured Output Constraint Design**: The project will explore the use of structured output constraints, such as JSON Schema, to ensure that the generated outlines strictly adhere to predefined hierarchical specifications. This mechanism safeguards the outline's quality in dimensions like logical coherence, hierarchical reasonableness, and page-level granularity.
- **Multi-Model Horizontal Comparison**: The project will conduct a horizontal comparison of several mainstream large language models, such as Qwen, GLM, DeepSeek, and so on, on the outline generation task. This comprehensive analysis will cover aspects such as generation stability, output quality, and response efficiency, providing data-driven support for the final technology selection.

Knowledge Retrieval and Content Completion

This module focuses on automatically retrieving, filtering, and condensing external knowledge to supplement each PPT page with high-quality text, data, and supporting arguments. By integrating RAG and DeepResearch technologies, the system ensures that the completed content is both information-dense and concise.

- **RAG-Based Knowledge Retrieval Module**: The project will develop a multi-turn retrieval strategy based on query rewriting anchored to outline nodes. The system will use the topic and key points of each page from the generated outline as retrieval anchors, automatically rewriting them into search queries suitable for search engines or knowledge bases, acquiring external information highly relevant to the page's content.
- **DeepResearch Integration**: The project will introduce a DeepResearch mechanism to enable multi-turn query rewriting and cross-source information integration capabilities. Through iterative multi-turn retrieval, the system will progressively refine knowledge structures and acquire comprehensive data support for complex content generation tasks.
- **Content Condensation and Synthesis**: The project will investigate methods to intelligently condense and synthesize lengthy retrieved texts so they fit the presentation needs of a single PPT page. It will focus on extracting key data, distilling core ideas, and filtering supporting evidence, ensuring that the supplemented content remains both information-dense and concise.

End-to-End Evaluation System

This module establishes a specialized multi-dimensional assessment framework for PPT

outline and content quality. The evaluation approach adopts a dual-track strategy combining human assessment with automated metrics, ensuring the comprehensiveness and reliability of the assessment results.

- **Evaluation Metric Design**: Evaluation metrics will cover key aspects including structural soundness (e.g., chapter organization, slide granularity), information density (e.g., amount of relevant information per slide), and factual accuracy (e.g., proportion of correct statements based on reference sources). These metrics provide a systematic framework for assessing the quality of generated content.
- **Human Assessment**: The project will conduct small-scale peer evaluation to obtain subjective scores on generated outlines and content, focusing on clarity, completeness, and presentation readiness from a typical user's perspective.
- **Automated Evaluation**: The project will employ natural language processing metrics (e.g., ROUGE, BERTScore) and simple rule-based scoring criteria to enable objective, quantitative, and reproducible assessment of content quality across different system configurations.

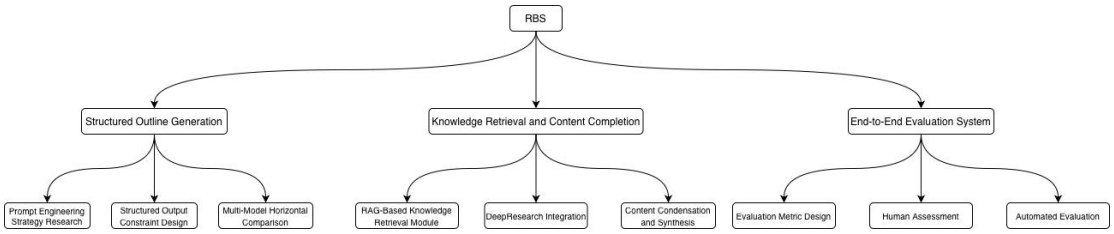

Figure 1 RBS Diagram

WBS

The Work Breakdown Structure (WBS) is a method of breaking down a project into smaller, manageable work units. It helps the project team refine specific work tasks and clarify each stage and step of the project. The WBS in this project covers the following core work areas, which are derived from the lowest-level requirements in the Requirements Breakdown Structure (RBS).

Detailed Work Breakdown Structure with Estimations (Total Budget: 220 hours)

1. Project Planning & Design (Total: 40 hrs) - [Status: Completed]				
1.1	Requirements gathering & technical feasibility analysis	All	12hrs	Completed
1.2	System architecture design (FastAPI + React + PostgreSQL)	Ma Xiaolong	10hrs	Completed
1.3	Database modeling (pgvector + Redis Stream + OSS)	Tang Chengyang	10hrs	Completed
1.4	Define 4-level PPT outline JSON Schema specification	Zhang Ruiqi	8hrs	Completed
2. Core Module Development (Total: 130 hrs) - [Status: In Progress]				
2.1	**Backend Architecture Implementation**	**Ma Xiaolong**	**35hrs**	**In Progress**
2.1.1	Implement FastAPI basic routing & dependency injection	Ma Xiaolong	10hrs	Completed
2.1.2	Develop Session/Task asynchronous state machine logic	Ma Xiaolong	15hrs	In Progress
2.1.3	Integrate OSS raw file storage interfaces	Ma Xiaolong	10hrs	Pending

2.2	**Prompt Engineering & Outline Generation**	**Zhang Ruiqi**	**30hrs**	**In Progress**
2.2.1	Draft intent_classifier.txt intent recognition prompt	Zhang Ruiqi	8hrs	Completed
2.2.2	Optimize outline generation prompts based on JS Schema	Zhang Ruiqi	10hrs	In Progress
2.2.3	API integration & comparative testing for multiple models	Zhang Ruiqi	12hrs	Pending
2.3	**RAG Retrieval Module**	**Tang Chengyang**	**35hrs**	**In Progress**
2.3.1	Configure Redis Stream asynchronous processing queue	Tang Chengyang	10hrs	Completed
2.3.2	Implement vector indexing and similarity search using pgvector	Tang Chengyang	15hrs	In Progress
2.3.3	Develop multi-format document parsing plugins	Tang Chengyang	10hrs	Pending
2.4	**Frontend UI Development**	**Xie Jingcheng**	**30hrs**	**In Progress**
2.4.1	Set up React + TypeScript + Vite basic scaffolding	Xie Jingcheng	8hrs	Completed
2.4.2	Develop frontend SSE (Server-Sent Events) streaming client	Xie Jingcheng	12hrs	In Progress
2.4.3	Build outline editing and preview interaction components	Xie Jingcheng	10hrs	Pending
3. Integration, Evaluation & Delivery (Total: 50 hrs) - [Status: Pending]				
3.1	Script end-to-end automated evaluation (ROUGE/BERTScore)	Xie Jingcheng	15hrs	Pending
3.2	DeepResearch search workflow integration	Tang Chengyang	15hrs	Pending
3.3	System integration testing & bug fixing	Ma Xiaolong	10hrs	Pending
3.4	Final prototype deployment & technical documentation delivery	All	10hrs	Pending

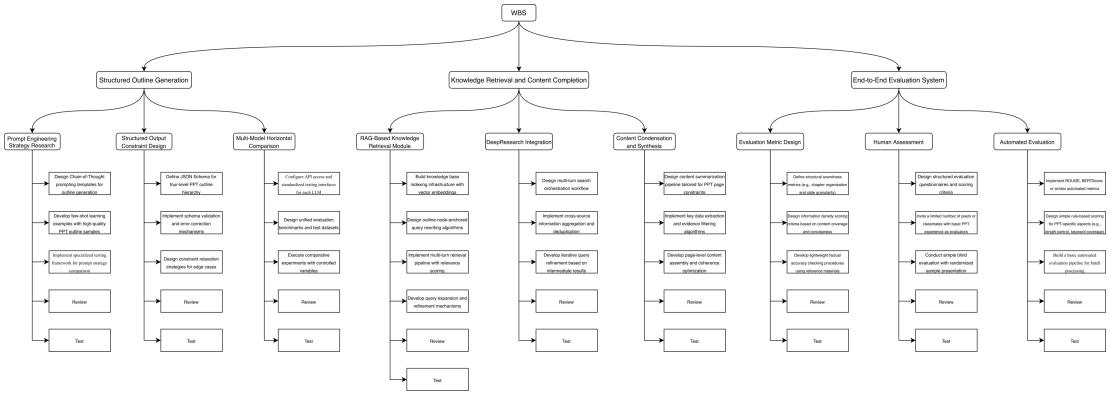

Figure 2 MBS Diagram

Gantt Chart

The development process for the Intelligent PPT Outline Generation and Content Completion System is organized into several distinct phases, each focusing on a key aspect of the project lifecycle.

Current Status Update (Sprint 1 Review) The project has successfully concluded its first biweekly sprint and is proceeding on track.

Phase 1: Project Planning & System Design (Mar 30 - Apr 12): [100% Completed]. The team has finalized the technology stack (React/FastAPI/PostgreSQL) and completed detailed architectural designs. The core JSON Schema specifications for the four-level outline (chapter,

page, points, notes) have been locked, laying a solid foundation for structured outputs.

Phase 2: Parallel Module Development (Started Apr 6): [In Progress].

- Frontend: Vite architecture is established; currently overcoming SSE streaming display challenges.
- Backend: FastAPI basic framework and routing are operational.
- RAG: Redis Stream asynchronous queue is configured; pgvector storage debugging is underway.
- Prompt Engineering: The intent recognition logic (intent_classifier) has passed initial verification.

Upcoming Forecast: Starting April 13, the team will fully initiate the *DeepResearch Integration* and *Multi-Model Comparison* modules. The final delivery date remains firmly scheduled for June 14, 2026.

Planning (2026-03-30 to 2026-04-03)

The project kicks off with a thorough requirements gathering phase. During this period, the team collaborates to define project objectives, scope, and essential features, establishing a solid foundation for all subsequent work. Team members conduct literature review on LLM-based content generation, RAG technologies, and DeepResearch approaches, identifying the state-of-the-art techniques and potential technical challenges.

System Design (2026-04-03 to 2026-04-06)

In this phase, the overall system architecture is designed. Decisions regarding the technology stack, API interfaces, data flow pipelines, and module integration points are made to ensure that the platform can support multi-model invocation, knowledge retrieval, and end-to-end evaluation workflows. The four-level outline schema (chapters, pages, key points, notes) is formally defined, and the JSON Schema specification is drafted.

Module Development (Starting 2026-04-06)

Multiple modules are developed concurrently to expedite the project:

Prompt Engineering and Outline Generation Module (2026-04-06 to 2026-05-04): Designs and implements various prompt strategies (Chain-of-Thought, few-shot learning) and structured output constraints (JSON Schema), conducts testing to identify optimal configurations for generating high-quality, four-level PPT outlines.

RAG Knowledge Retrieval Module (2026-04-06 to 2026-05-04): Builds the knowledge base indexing infrastructure with vector embeddings, implements outline-node-anchored query rewriting algorithms, and develops multi-turn retrieval pipelines with relevance scoring.

DeepResearch Integration Module (2026-04-13 to 2026-05-11): Designs multi-turn search orchestration workflows, implements cross-source information aggregation and deduplication, and develops iterative query refinement mechanisms based on intermediate results.

Content Condensation and Synthesis Module (2026-04-20 to 2026-05-11): Develops content summarization pipelines tailored for PPT page constraints, implements key data extraction algorithms, and builds page-level content assembly and coherence optimization.

Multi-Model Comparison Module (2026-04-13 to 2026-05-04): Configures API access for LLM, designs unified evaluation benchmarks, and executes comparative experiments with controlled variables.

Evaluation System Development (2026-05-04 to 2026-05-18)

The evaluation framework is built in parallel with module refinement. Evaluation metrics are defined, human assessment procedures are designed, automated evaluation pipelines using ROUGE, BERTScore, and custom scoring rules are implemented, and evaluator recruitment and training are conducted.

Integration and Testing (2026-05-11 to 2026-05-25):

All modules are integrated and tested to ensure seamless interaction and system stability. The outline generation module, RAG retrieval module, DeepResearch module, and content condensation module are connected into a unified end-to-end pipeline. This phase is crucial for identifying and resolving any inconsistencies or bugs before moving forward.

Experimental Validation and Analysis (2026-05-18 to 2026-06-01):

Comprehensive experiments are conducted to validate the system's effectiveness. Comparative experiments between pure LLM generation and RAG-augmented generation are executed. Multi-scenario tests covering short topic inputs, long document inputs, and specialized domain topics are performed. Human evaluation sessions and automated metric analyses are carried out to produce the technical validation report.

Finalization and Deployment (2026-06-01 to 2026-06-14)

The project concludes with final testing, bug fixing, and thorough documentation. The technical validation report is finalized, including experimental comparisons and analyses of different technical approaches. The prototype system is packaged and deployed, and all project deliverables are submitted, marking the successful completion on June 14.

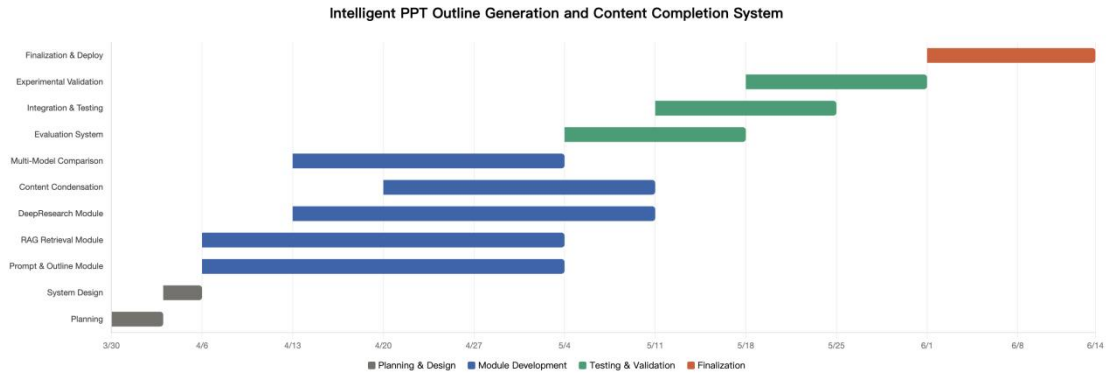

Figure 3 Gantt Chart

Critical Path Diagram

This critical path diagram meticulously illustrates the essential sequence of activities in the development and deployment of the Intelligent PPT Outline Generation and Content Completion System, revealing the dependencies between various project phases.

Initially, the project commences with the requirements gathering phase, where the functional needs and objectives of the system are clearly defined. This is followed by the system architecture design phase, which lays the foundation for the system's overall structure including the four-level outline schema, API interfaces, and data flow pipelines.

Subsequently, the process moves to the prompt engineering and outline generation phase, a crucial component for the core generation capability of the system. This phase involves designing and testing various prompt strategies (Chain-of-Thought, few-shot learning) and implementing structured output constraints (JSON Schema). In parallel, the RAG knowledge retrieval module is developed, providing the essential external knowledge support for content completion.

Following this, the project progresses to the DeepResearch integration phase, enabling multi-turn query rewriting and cross-source information aggregation. The content condensation and synthesis module is then developed, transforming retrieved knowledge into presentation-ready page content. The multi-model comparison module is also developed during this phase, ensuring data-driven model selection decisions.

After completing the development of these key modules, the project enters the evaluation system development phase, where automated metrics (ROUGE, BERTScore) and human assessment procedures are established. Next, through system integration and testing, all modules are connected into a unified end-to-end pipeline to ensure seamless collaboration.

The experimental validation and analysis phase follows, conducting comprehensive comparative experiments (pure LLM vs. RAG-augmented generation) and multi-scenario tests. Finally, the finalization and deployment phase involves completing the technical validation report, packaging the prototype system, and submitting all project deliverables, preparing the system for

presentation.

This critical path diagram not only clearly demonstrates the temporal dependencies between each phase but also helps us identify the shortest path to project completion, providing robust support for rational project scheduling and timely delivery. Through this systematic path management, we can efficiently advance the development and deployment of the system.

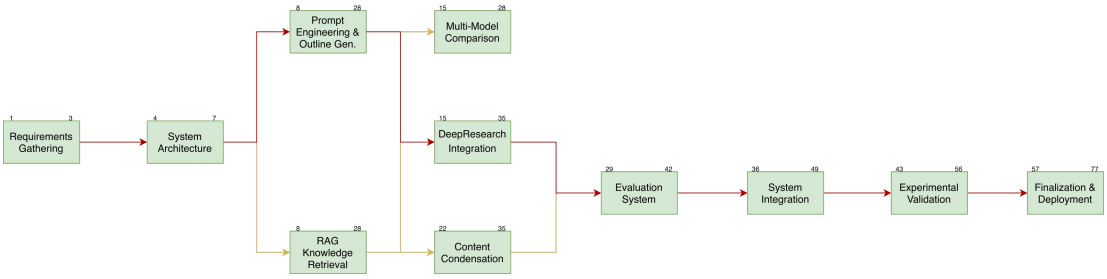

Figure 4 Critical Path Diagram